Complete Bit Manipulation Theory Notes — Interview Revision Guide

> **Purpose:** Conceptual, proof-based, intuition-driven revision notes.
> **Style:** Handwritten-revision flavour. No heavy code — only binary reasoning.
> **Scope:** Beginner → Advanced. Every "why" answered.

---

§1 — Core Binary Fundamentals

1.1 The Binary Number System

Every number we store in a computer is a sequence of **bits** (binary digits), each either `0` or `1`.

Positional Value Formula

Each bit position `i` (counting from the right, starting at 0) has a **weight** of 2^i.

$$V = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

Example — 13 in binary:

```
Position:   3    2    1    0
Bits:       1    1    0    1
Weight:     8    4    2    1
Value:      8 +  4 +  0 +  1 = 13
```

Key insight: Binary is just base-2 positional notation. The same idea as decimal (base-10), but each column is a power of 2 instead of a power of 10.

1.2 Decimal ↔ Binary Conversion

Decimal → Binary (Repeated Division by 2)

Divide by 2, record remainders bottom-to-top.

```
13 ÷ 2 = 6  remainder 1  ← LSB (rightmost)
 6 ÷ 2 = 3  remainder 0
 3 ÷ 2 = 1  remainder 1
 1 ÷ 2 = 0  remainder 1  ← MSB (leftmost)

Read remainders upward → 1101₂
```

Why it works: Each division extracts one bit via the **Euclidean Division Algorithm**:
$D = Q \times 2 + R$, where $R \in \{0, 1\}$ is exactly the current least significant bit.

Binary → Decimal (Power Summation)

```
10110₂ = 1·2⁴ + 0·2³ + 1·2² + 1·2¹ + 0·2⁰
       = 16  +  0  +  4  +  2  +  0
       = 22
```

How many bits to represent N?

$$\text{bits needed} = \lfloor \log_2 N \rfloor + 1$$

1.3 Bit Positions & Indexing

```
Bit:      1  0  1  1  0  1  0  0
Index:    7  6  5  4  3  2  1  0
          ↑                    ↑
         MSB                  LSB
```

Term	Meaning
LSB (Least Significant Bit)	Bit at position 0 (rightmost). Decides odd/even
MSB (Most Significant Bit)	Highest-weight bit. In signed numbers, it's the sign bit

Indexing is 0-based from the right. Position `k` has weight 2^k.

1.4 Signed vs Unsigned Representation

Unsigned (n bits)

All bits contribute positively.
Range: $[0,\ 2^n - 1]$

For 8 bits: $[0,\ 255]$

Signed — Two's Complement (n bits)

The MSB has **negative weight**:

$$V = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

Range: $[-2^{n-1},\ 2^{n-1} - 1]$

For 8 bits: $[-128,\ 127]$

Why the range is asymmetric: There's one more negative number than positive because `0` takes a spot on the positive side.

1.5 One's Complement & Two's Complement

One's Complement of x

Flip every bit.

```
 x     = 0 1 0 1 1 0 0 0  (88)
~x     = 1 0 1 0 0 1 1 1  (one's complement)
```

Problem: One's complement gives TWO representations of zero:

```
+0 = 00000000
-0 = 11111111    ← "Double Zero" problem
```

Two's Complement of x

$$-x = \sim x + 1$$

Steps: Flip all bits, then add 1.

```
 x     = 00000101   (+5)
~x     = 11111010   (flip)
~x + 1 = 11111011   (-5 in two's complement)
```

Verification: $+5 + (-5)$

```
  0 0 0 0 0 1 0 1
+ 1 1 1 1 1 0 1 1
 ───────────────
1 0 0 0 0 0 0 0 0  ← carry overflows out, remaining = 0 ✓
```

Why Two's Complement?

Reason	Explanation
No double zero	Only one representation of 0
Same hardware for + and −	The ALU doesn't need separate adder and subtractor circuits
Additive inverse works naturally	$x + (-x)$ overflows to exactly 0
Comparison is simple	MSB directly tells you the sign

1.6 Overflow — Intuition

For signed `n`-bit integers, values "wrap around" modulo 2^n:

```
8-bit signed:
  127 + 1 = -128  (positive overflow → wraps to most negative)
 -128 - 1 =  127  (negative overflow → wraps to most positive)
```

Detection rule: Overflow occurs when the **carry into** and **carry out of** the sign bit differ.

Intuition: Think of a clock. After 12 comes 1 again. Similarly, after 0111...1 (max positive) comes 1000...0 (most negative).

---

§2 — Bitwise Operators (Pure Theory)

2.1 AND (&) — "The Filter"

Truth Table

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Output is 1 only when BOTH inputs are 1.

Mathematical Interpretation

AND performs **bit-level multiplication**: $A \cdot B$

Binary Intuition

AND is a **filter/mask**: it lets through only bits that are "on" in both operands.

```
  1 1 0 1 0 1 1 0   (214)
& 0 0 1 1 1 1 0 0   (60)  ← the "mask"
─────────────────
  0 0 0 1 0 1 0 0   (20)  ← only bits where BOTH are 1 survive
```

Use cases:

- **Extract bits:** `x & mask` keeps only the bits where `mask` has 1s
- **Check a bit:** `x & (1 << k)` — non-zero iff bit `k` is set
- **Clear bits:** `x & ~mask` turns off bits where `mask` has 1s

2.2 OR (|) — "The Setter"

Truth Table

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Output is 1 when AT LEAST ONE input is 1.

Mathematical Interpretation

$A + B - A \cdot B$ (inclusion-exclusion at the bit level).

Binary Intuition

OR is a **setter/painter**: it turns ON bits.

```
  1 0 1 0 0 0 0 0   (160)
| 0 0 0 0 1 1 0 1   (13)
─────────────────
  1 0 1 0 1 1 0 1   (173)  ← bits from EITHER operand are set
```

Use case: `x | (1 << k)` sets bit `k` to 1, regardless of its current value.

2.3 XOR (^) — "The Toggler / Difference Detector"

Truth Table

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

Output is 1 when inputs DIFFER.

Mathematical Interpretation

$A \oplus B = A\bar{B} + \bar{A}B$ (exclusive disjunction).
Also: **addition without carry** $\rightarrow A + B \mod 2$.

Binary Intuition

XOR is a **toggler**: where the "mask" has 1, it flips the bit. Where 0, it preserves.

```
  1 0 1 1 0 1 0 0   (180)
^ 0 0 1 1 1 1 0 0   (60)
─────────────────
  1 0 0 0 1 0 0 0   (136)  ← bits FLIP where mask is 1
```

Why XOR is powerful: It's the **only** bitwise operator that's reversible.
If `C = A ⊕ B`, then `A = C ⊕ B` and `B = C ⊕ A`.

2.4 NOT (~) — "The Inverter"

Truth Table

A	~A
0	1
1	0

Flips every bit.

Mathematical Interpretation

In two's complement: $\sim x = -(x+1)$

Proof:

$$x + \sim x = \underbrace{111\ldots1}_{n \text{ bits}} = -1 \text{ (in two's complement)}$$

$$\therefore \sim x = -1 - x = -(x+1)$$

Binary Intuition

```
 x  = 0 0 0 0 1 0 1 0   (+10)
~x  = 1 1 1 1 0 1 0 1   (-11 in two's complement)
```

Warning: NOT on a positive number gives a negative number (and vice versa), because the sign bit flips too.

2.5 Left Shift (<<) — "Multiply by 2^k"

`x << k` shifts all bits left by `k` positions, filling vacated positions with 0.

```
x     = 0 0 0 0 1 1 0 1   (13)
x << 2= 0 0 1 1 0 1 0 0   (52)
```

Why it's multiplication by 2^k

Shifting left by 1 moves every bit to the next higher weight position:

$$b_i \cdot 2^i \longrightarrow b_i \cdot 2^{i+1} = b_i \cdot 2^i \cdot 2$$

So shifting left by `k`:

$$x \ll k = x \cdot 2^k$$

Overflow warning: Bits that shift beyond the MSB are lost. $200 \ll 1$ in 8-bit unsigned = $400 \mod 256 = 144$ (overflow).

2.6 Right Shift (>>) — "Divide by 2^k"

`x >> k` shifts all bits right by `k` positions.

```
x     = 0 0 1 1 0 1 0 0   (52)
x >> 2= 0 0 0 0 1 1 0 1   (13)
```

Why it's division by 2^k (for positive numbers)

Shifting right by 1 moves every bit to the next lower weight position:

$$b_i \cdot 2^i \longrightarrow b_i \cdot 2^{i-1} = b_i \cdot 2^i \div 2$$

So: $x \gg k = \lfloor x/2^k \rfloor$ (floor division, bits that fall off are lost).

What happens for negative numbers?

Two types of right shift:

Type	Fill bit	Effect
Logical shift	Always fills with 0	Treats as unsigned
Arithmetic shift	Fills with the sign bit (MSB)	Preserves sign

```
x     = 1 1 0 0 1 0 0 0   (-56 signed)

Arithmetic >> 2:
        1 1 1 1 0 0 1 0   (-14) ← sign preserved, fills with 1

Logical >> 2:
        0 0 1 1 0 0 1 0   (+50) ← sign NOT preserved, fills with 0
```

Key fact: Arithmetic right shift of a negative number rounds toward $-\infty$ (not toward zero).

The infinite shift trap: Arithmetic right-shifting `-1` (`11111111`) by any amount still gives `11111111` = `-1`. You can never shift away all the 1s. This causes infinite loops if you shift until zero.

---

§3 — XOR: Deep Properties & Proofs

XOR forms an **Abelian Group** over $\{0,1\}^n$. This is the theoretical foundation for all XOR-based interview problems.

3.1 The Four Properties

Property 1: Commutative — $A \oplus B = B \oplus A$

XOR checks "are bits different?" — order doesn't matter.

Property 2: Associative — $(A \oplus B) \oplus C = A \oplus (B \oplus C)$

We can XOR any sequence in any grouping and get the same result.

Property 3: Identity — $A \oplus 0 = A$

Proof (bit-level):

A	0	A ⊕ 0
0	0	0
1	0	1

XOR with 0 preserves every bit. Zero is the identity element.

Property 4: Self-Inverse — $A \oplus A = 0$

Proof (bit-level):

A	A	A ⊕ A
0	0	0
1	1	0

Same bits → no difference → result is 0.

Boolean algebra proof:

$$A \oplus A = A\bar{A} + \bar{A}A = 0 + 0 = 0$$

3.2 The Cancellation Property (KEY)

$$X \oplus Y \oplus X = Y$$

4-step derivation:

```
  X ⊕ Y ⊕ X
= X ⊕ X ⊕ Y       ← (1) Commutativity: reorder
= (X ⊕ X) ⊕ Y     ← (2) Associativity: group
= 0 ⊕ Y           ← (3) Self-Inverse: X ⊕ X = 0
= Y               ← (4) Identity: 0 ⊕ Y = Y
```

This is WHY XOR cancels duplicates.

3.3 Why XOR Works in Interview Problems

Problem 1: Single Non-Repeating Number

> Array where every element appears **twice** except one. Find it.

XOR all elements:

$$a_1 \oplus a_1 \oplus a_2 \oplus a_2 \oplus \cdots \oplus x = 0 \oplus 0 \oplus \cdots \oplus x = x$$

Every duplicate pair cancels to 0 (Self-Inverse + Associativity). The lone element survives (Identity).

Example: `[2, 3, 5, 3, 2]`

```
  2 ⊕ 3 ⊕ 5 ⊕ 3 ⊕ 2
= (2 ⊕ 2) ⊕ (3 ⊕ 3) ⊕ 5
=    0     ⊕    0     ⊕ 5
= 5 ✓
```

Problem 2: Missing Number

> Array `[0, 1, 2, ..., n]` with one number missing. Find it.

XOR all array elements with XOR of `0` through `n`:

$$\bigoplus_{i=0}^{n} i \oplus \bigoplus_{\text{arr}} a_j$$

Every number that appears in both sets cancels. The missing number survives.

Problem 3: Two Non-Repeating Numbers

> Array where every element appears twice except **two** numbers `x` and `y`. Find both.

Step 1: XOR all elements → get `x ⊕ y` (all pairs cancel).

Step 2: Since $x \neq y$, we know $x \oplus y \neq 0$. At least one bit position differs between `x` and `y`.

Step 3: Find any set bit in `x ⊕ y` (use rightmost set bit — see §5). Call this bit position `p`.

Step 4 (The Partition): Split ALL array elements into two groups:

- Group A: bit `p` is 1
- Group B: bit `p` is 0

Why this works:

- `x` and `y` differ at bit `p`, so they land in **different groups**
- Every duplicate pair has the **same** bit `p`, so both copies land in the **same** group
- XOR of each group gives one unique number

This is the **Binary Sieve**. (See §5 for the full rightmost-bit mechanism.)

---

§4 — Important Bit Manipulation Identities (Mathematical Proofs)

4.0 What Happens When You Subtract 1 from a Binary Number?

This is the **foundation** of almost every bit identity. Understand this deeply.

The Borrow Ripple

Consider any number `x` in binary. It has the form:

```
x = [prefix] 1 [trailing zeros]
     ↑        ↑      ↑
     A        p      B = 000...0
```

- **A** = all bits to the left of the rightmost `1`
- **p** = the rightmost `1` (the pivot)
- **B** = all trailing zeros to the right of `p`

When we compute x − 1:

Subtracting 1 from the LSB side triggers a **borrow ripple**:

1. The trailing zeros (`B` = `000...0`) can't give up a 1, so they borrow and become `111...1`
2. The rightmost `1` (`p`) donates, becoming `0`
3. Everything to the left (`A`) is untouched

```
x     = [ A ] 1 [000...0]       ← original
x - 1 = [ A ] 0 [111...1]       ← after subtraction
              ↑      ↑
         p flips   trailing zeros flip
```

State Transformation Table

Zone	Bits in x	Bits in (x−1)	What happened
Prefix (A)	unchanged	unchanged	No borrow reached here
Pivot (p)	1	0	Donated to borrow
Suffix (B)	000...0	111...1	Borrowed and flipped

This is the SINGLE most important insight in bit manipulation.

4.1 Identity: `x & (x - 1)` Removes the Rightmost Set Bit

Binary Proof (3-Zone Analysis)

```
x     = [ A ] 1 [000...0]
x - 1 = [ A ] 0 [111...1]
```

Now AND them:

Zone	x	x−1	x & (x−1)	Why
Prefix A	A	A	A	$A \,\&\, A = A$ (Idempotent law)
Pivot	1	0	0	$1 \,\&\, 0 = 0$ (Annihilation)
Suffix B	000...0	111...1	000...0	$0 \,\&\, 1 = 0$ for each bit

Result:

```
x & (x-1) = [ A ] 0 [000...0]
```

The rightmost `1` and everything to its right vanish. **Exactly one set bit removed.**

Walkthrough: x = 20

```
x     = 10100       (20)
x - 1 = 10011       (19)
         ↑↑↑↑↑
AND:   = 10000       (16)  ← rightmost 1 (at position 2) removed ✓
```

Walkthrough: x = 12

```
x     = 01100       (12)
x - 1 = 01011       (11)

AND:   = 01000       (8)   ← rightmost 1 (at position 2) removed ✓
```

4.2 Identity: `x & (-x)` Extracts the Rightmost Set Bit

How −x is formed (Two's Complement)

$$-x = \sim x + 1$$

Step 1 — Invert all bits ($\sim x$):

```
x  = [ A ] 1 [000...0]
~x = [~A ] 0 [111...1]
```

Step 2 — Add 1 to $\sim x$:

Adding 1 to `[~A] 0 [111...1]` causes the trailing `1`s to carry-ripple:

- `111...1` + 1 → `000...0` (with carry)
- The carry hits the `0` at the pivot, turning it to `1`
- `~A` stays unchanged (carry stops)

```
-x = [~A ] 1 [000...0]
```

Now AND with original x:

```
x  = [ A ] 1 [000...0]
-x = [~A ] 1 [000...0]
```

Zone	x	−x	x & (−x)	Why
Prefix	A	~A	000...0	$A \& \sim A = 0$ (Complementation)
Pivot	1	1	1	$1 \& 1 = 1$
Suffix	000...0	000...0	000...0	$0 \& 0 = 0$

Result:

```
x & (-x) = [000...0] 1 [000...0]
```

Only the rightmost set bit survives. Everything else is zeroed out.

Walkthrough: x = 12

```
x    = 01100       (12)
-x   = 10100       (-12 in two's complement)

AND: = 00100       (4)  ← rightmost set bit isolated ✓

(Position 2, weight 2² = 4)
```

4.3 Identity: `x & ~(x - 1)` Also Extracts the Rightmost Set Bit

Proof

```
x     = [ A ] 1 [000...0]
x - 1 = [ A ] 0 [111...1]
~(x-1)= [~A ] 1 [000...0]      ← flip every bit
```

Now AND:

```
  x       = [ A ] 1 [000...0]
& ~(x-1) = [~A ] 1 [000...0]
```

Zone	Result	Why
Prefix	000...0	$A \& \sim A = 0$
Pivot	1	$1 \& 1 = 1$

Zone	Result	Why
Suffix	000...0	$0 \& 0 = 0$

Same result: only the rightmost set bit survives.

4.4 Why `x & (-x)` ≡ `x & ~(x - 1)` — Proof of Equivalence

Both formulas produce the same result because:

$$-x = \sim x + 1$$

But also, by the mechanics above, $-x$ and $\sim (x-1)$ produce the **exact same bit pattern**:

$$\sim (x-1) = -x$$

Proof:

$$\sim (x-1) = -(x-1) - 1 = -x + 1 - 1 = -x$$

(Using the identity $\sim y = -(y+1)$, substitute $y = x - 1$.)

Therefore:

$$x \& \sim (x-1) = x \& (-x)$$

They are **algebraically identical**, not just coincidentally same. Use whichever makes the code clearer.

---

§5 — Rightmost Set Bit: Deep Conceptual Explanation

5.1 Binary Transformation After Subtraction — Visual Walkthrough

Let's trace through `x = 52` (binary `110100`):

```
x     = 1 1 0 1 0 0
                ↑
            rightmost 1 (position 2)

Subtract 1 (borrow ripple):
            1 1 0 1 0 0
          - 0 0 0 0 0 1
          ─────────────
Step 1: Position 0 is 0, can't subtract → borrow from position 1
Step 2: Position 1 is 0, can't lend → borrow from position 2
Step 3: Position 2 is 1 → LENDS. Becomes 0.
Step 4: Position 1 receives borrow → becomes 1 (but gives to pos 0)
Step 5: Position 0 receives borrow → becomes 1, subtraction done

x - 1 = 1 1 0 0 1 1
```

Compare bit by bit:

```
x     = 1 1 0 [1] 0 0
x - 1 = 1 1 0 [0] 1 1
                ↑   ↑↑
          pivot flips  trailing zeros flip
```

Everything left of the rightmost `1` is IDENTICAL.
The rightmost `1` and everything to its right are COMPLEMENTED.

5.2 Why This Identity Helps Partition Arrays

In the "Two Unique Numbers" problem (§3.3), after XOR gives us $xr = x \oplus y$:

```
xr = x ⊕ y = some non-zero value
```

The rightmost set bit of `xr` tells us the **lowest bit position where x and y differ**.

```
Isolate it:  mask = xr & (-xr)
```

Now partition the array: for every element `a`:

- If `a & mask ≠ 0` → goes to Group 1
- If `a & mask = 0` → goes to Group 2

Guarantee 1: `x` and `y` are in different groups (they differ at this bit).
Guarantee 2: Every duplicate pair goes to the same group (identical numbers have identical bits).

XOR within each group → one unique number per group.

---

§6 — Counting Set Bits (Population Count / Hamming Weight)

6.1 Brian Kernighan's Algorithm — Theory

Core idea: Repeatedly apply `x = x & (x - 1)` until `x = 0`. Each application removes exactly one set bit (proven in §4.1). Count the iterations.

Why it removes EXACTLY one bit per step

From the 3-zone proof:

- **Prefix:** unchanged (idempotent AND)
- **Pivot:** 1 → 0 (one bit removed)
- **Suffix:** already all zeros, stays zero

No other bits are affected. Exactly one bit dies per iteration.

Walkthrough: x = 13 (binary 1101)

```
Step 1:  1101 & 1100 = 1100   (removed bit 0)  count = 1
Step 2:  1100 & 1011 = 1000   (removed bit 2)  count = 2
Step 3:  1000 & 0111 = 0000   (removed bit 3)  count = 3
x = 0 → stop. Answer: 3 set bits. ✓
```

Time Complexity

- **Best case:** $O(1)$ — number is 0 (no iterations)
- **Worst case:** $O(n)$ — all n bits are set (e.g., 255 = `11111111` → 8 iterations)
- **Average/Typical:** $O(K)$ where K = number of set bits (Hamming weight)

Key advantage over naive approach: Naive scans all n bit positions ($O(n)$). Kernighan's skips the zeros entirely. For sparse numbers (few 1s), it's dramatically faster.

Approach	Complexity	Iterates over
Naive (check each position)	$O(n)$ where n = bit width	Every bit position

Approach	Complexity	Iterates over
Kernighan's	$O(K)$ where K = popcount	Only set bits
Hardware POPCNT	$O(1)$	Single instruction

---

§7 — Power of Two: Concept & Proof

7.1 The Single-Bit Property

A number is a power of 2 if and only if its binary representation has **exactly one** `1` bit.

```
1   = 00000001  ← 2⁰  ✓
2   = 00000010  ← 2¹  ✓
4   = 00000100  ← 2²  ✓
8   = 00001000  ← 2³  ✓
16  = 00010000  ← 2⁴  ✓

6   = 00000110  ← NOT power of 2 (two 1-bits)
```

7.2 Why `n & (n - 1) == 0` for Powers of Two

Proof for $n = 2^k$

```
n     = 0...0 1 0...0     (single bit at position k)
n - 1 = 0...0 0 1...1     (borrow ripple: pivot→0, trailing→1s)
```

There is **no prefix** (everything left of position `k` is zero). So:

```
n     = 0...0 1 0...0
n - 1 = 0...0 0 1...1
AND   = 0...0 0 0...0 = 0 ✓
```

The single `1` in `n` becomes `0` in `n-1`. Below that, `n` has all `0`s and `n-1` has all `1`s. AND of opposite bits = 0 everywhere.

They have zero bits in common → result is 0.

Proof that it FAILS for non-powers of two

If `n` has more than one `1` bit, then `n & (n-1)` removes only the **rightmost** `1`. The other `1` bits survive. Result ≠ 0.

Example: n = 6

```
n     = 110
n - 1 = 101
AND   = 100 ≠ 0   ← second `1` survives. Not a power of 2.
```

7.3 Edge Case: n = 0

```
0     = 000...0
0 - 1 = 111...1    (underflow → all 1s in unsigned, or -1 in signed)
AND   = 000...0 = 0
```

`0 & (-1) = 0` — this **wrongly passes** the test! But 0 is NOT a power of 2.

Correct check:

$$n > 0 \quad \text{AND} \quad n \,\&\, (n-1) = 0$$

The `n > 0` guard eliminates the false positive.

---

§8 — Bitmasking: Complete Theory

8.1 What is a Bitmask?

A **bitmask** is an integer whose binary representation encodes a **set**.

Each bit position `i` represents an element:

- Bit `i` = 1 → element `i` is **included**
- Bit `i` = 0 → element `i` is **excluded**

```
Set = {apple, banana, cherry}    (indexed 0, 1, 2)

mask = 101₂ = 5  →  {apple, cherry}      (bits 0 and 2 set)
mask = 011₂ = 3  →  {apple, banana}      (bits 0 and 1 set)
mask = 111₂ = 7  →  {apple, banana, cherry}  (all bits set)
mask = 000₂ = 0  →  {}  (empty set)
```

8.2 Why Total Subsets = 2^n

For a set of `n` elements, each element has 2 choices: **in** or **out**. By the multiplication principle:

$$\text{Total subsets} = \underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ times}} = 2^n$$

A bitmask of `n` bits naturally enumerates all 2^n possibilities: every integer from `0` to $2^n - 1$ represents exactly one unique subset.

```
n = 3 elements → 2³ = 8 subsets

mask=0 (000): {}           mask=4 (100): {c}
mask=1 (001): {a}          mask=5 (101): {a,c}
mask=2 (010): {b}          mask=6 (110): {b,c}
mask=3 (011): {a,b}        mask=7 (111): {a,b,c}
```

8.3 Set Operations via Bitmasks

Set Operation	Bitmask Operation	Example
Union $A \cup B$	`A | B`	`101 | 011 = 111`
Intersection $A \cap B$	`A & B`	`101 & 011 = 001`
Difference $A \setminus B$	`A & ~B`	`101 & ~011 = 100`
Complement $\bar{A}$	`~A & ((1<<n)-1)`	Flip within n bits
Membership $i \in A$?	`A & (1 << i)`	Non-zero → yes
Add element $A \cup \{i\}$	`A | (1 << i)`	Set bit `i`
Remove element $A \setminus \{i\}$	`A & ~(1 << i)`	Unset bit `i`
Toggle element	`A ^ (1 << i)`	Flip bit `i`
Size (popcount)	Brian Kernighan's / POPCNT	Count set bits

8.4 The Inclusion-Exclusion Interpretation

Each element is either included or excluded — binary choice. The bitmask is the **characteristic vector** of the subset. This connects combinatorics directly to binary arithmetic.

Iterating `mask` from `0` to $2^n - 1$ is equivalent to iterating over the **power set** of $\{0, 1, \ldots, n-1\}$.

8.5 How Bitmasking Helps in Algorithms

(a) Bitmask DP (State Compression)

Problem: When the state involves "which elements have been used/visited," a subset of n elements can be represented by an n-bit mask.

Instead of: An array of n booleans (needs $O(n)$ space per state, $O(n)$ comparison)
Use: A single integer (needs $O(1)$ space per state, $O(1)$ comparison, $O(1)$ hashing)

Example — TSP (Travelling Salesman Problem):

$$DP[\text{mask}][i] = \text{minimum cost to visit the cities in `mask`, ending at city } i$$

- `mask` = which cities have been visited (bitmask)
- Transition: try adding an unvisited city `j`: `if !(mask & (1 << j))`
- New state: `DP[mask | (1 << j)][j]`
- Complexity: $O(2^n \cdot n^2)$ — tractable for $n \leq 20$

(b) Graph Problems

- Subset enumeration for cliques, independent sets
- State tracking in BFS/DFS (which nodes visited)

(c) Why the "Exponential Wall" Matters

n	2^n	Feasibility
10	1,024	Very fast
15	32,768	Fast
20	~10^6	Feasible
25	~3.3×10^7	Borderline
30	~10^9	Too slow

Interview heuristic: If $n \leq 20$ in the constraints, consider bitmask DP.

8.6 Iterating Over All Subsets of a Mask

To enumerate all submasks of a mask `m`:

```
s = m
while s > 0:
    process(s)
    s = (s - 1) & m
```

Why it works:

- `s - 1` decrements to the "previous" number (borrow ripple)
- `& m` forces the result back into the domain of `m`'s set bits
- This visits every subset of `m` in decreasing order

Total complexity over all masks: $O(3^n)$

Why 3^n? Each of the n elements has 3 states: (1) not in the outer mask, (2) in the outer mask but not the submask, (3) in both. By the multiplication principle: 3^n total.

§9 — Advanced Concepts

9.1 Bit DP — State Compression Idea

Core concept: Replace a k-dimensional boolean state with a single integer.

Why it works:

- A set of n items → 2^n possible subsets → each uniquely encoded as an n-bit integer
- DP table becomes `DP[mask]` instead of `DP[visited[0]][visited[1]]...[visited[n-1]]`
- **Dimension reduction:** If the DP also tracks the count of selected items, note that `popcount(mask)` gives that for free. So `DP[count][mask]` collapses to just `DP[mask]`.

Typical pattern:

```
For each mask from 0 to 2^n - 1:
    For each bit i set in mask:
        Try transitions to mask with bit i removed or added
```

Cache locality benefit: Integer masks fit in registers/L1 cache. Boolean arrays do not. This gives real speedups beyond just asymptotic complexity.

9.2 Maximum XOR — Concept & Intuition

Problem: Given an array, find two elements whose XOR is maximum.

Key insight — MSB dominance:

$$2^k > 2^{k-1} + 2^{k-2} + \cdots + 2^1 + 2^0 = 2^k - 1$$

Proof: $\sum_{j=0}^{k-1} 2^j = 2^k - 1 < 2^k$

What this means: A `1` at a higher bit position is worth more than ALL lower bits combined. So to maximize XOR, we must greedily set the highest possible bits to 1.

Greedy strategy: Process bits from MSB to LSB. At each bit position, try to make the XOR result have a `1`. If any pair of elements can achieve this (one has `0`, the other has `1` at this position), take it.

9.3 Binary Trie for XOR Optimization

A **Binary Trie** (Prefix Tree over bits) enables efficient maximum-XOR queries.

Structure

- Each node has at most 2 children: `0`-child and `1`-child
- Depth = number of bits (e.g., 32 for 32-bit integers)
- Each root-to-leaf path represents one number's binary form (MSB to LSB)

How it works for Maximum XOR

Insert: For each number, walk down from MSB to LSB, creating nodes as needed.

Query: For a number `x`, walk the trie greedily:

- At each level, try to go to the **opposite** bit (to make XOR = 1)
- If opposite child exists → take it (XOR bit = 1)
- If only same child exists → take it (XOR bit = 0)

The greedy path gives the number in the trie that produces maximum XOR with `x`.

Why greedy is optimal

Because of MSB dominance ($2^k > \sum_{j<k} 2^j$), getting a `1` at a higher position always beats any combination of lower bits. No backtracking needed.

Complexity: $O(n \cdot W)$ where W = word size (typically 32).

---

§10 — Common Interview Patterns

10.1 When to Think of XOR

Signal	Why XOR
"Find the number that appears odd times"	XOR cancels pairs
"Find missing number in range"	XOR range with array
"Find two unique numbers among duplicates"	XOR + partition by rightmost set bit
"Maximum XOR of pair"	Trie + greedy on bits
"No extra space, O(n) time" for finding duplicates	XOR doesn't need storage

10.2 Partitioning Numbers Using Rightmost Set Bit

When XOR gives a combined result and you need to split:

1. Compute `xr = total XOR`
2. Isolate: `mask = xr & (-xr)`
3. Partition: `element & mask == 0` vs `!= 0`
4. XOR within each partition

This works because identical elements always fall in the same partition.

10.3 Recognizing Hidden Power-of-Two Patterns

Look for:

- "Exactly one bit set" → `n & (n-1) == 0`
- "Size is power of 2" → suggests divide-and-conquer or bit tricks
- "Round up to next power of 2" → repeated OR-shift pattern

10.4 Detecting Bit Tricks in Constraints

Constraint Pattern	What it Suggests
$n \leq 20$	Bitmask DP ($2^{20} \approx 10^6$)
$n \leq 15$	Bitmask with submask enumeration ($3^{15} \approx 1.4 \times 10^7$)
Values up to 10^9 (fits 30 bits)	Bit-by-bit processing, tries
"Without extra space"	XOR trick or in-place bit manipulation
"O(n) time, O(1) space"	XOR or bitwise trick

---

§11 — Important Bit Tricks (Theory Only)

11.1 Check Even/Odd

Operation: `n & 1`

Why: The LSB determines parity.

- If LSB = 1 → number is odd (has a $2^0 = 1$ component)
- If LSB = 0 → number is even (no 2^0 component)

```
7 = 111  →  111 & 001 = 001 = 1  → odd ✓
8 = 1000 → 1000 & 0001 = 0000 = 0 → even ✓
```

Why it's better than modulo: Bitwise AND is a single CPU instruction vs. division for `% 2`.

11.2 Swap Two Numbers Using XOR

Three-step transformation:

```
a = a ⊕ b
b = a ⊕ b
a = a ⊕ b
```

Proof (tracking values):

Let original values be A and B.

Step	a becomes	b becomes	Reasoning
1	$A \oplus B$	B	Store XOR in a
2	$A \oplus B$	$(A \oplus B) \oplus B = A$	Cancel B: $B \oplus B = 0$
3	$(A \oplus B) \oplus A = B$	A	Cancel A: $A \oplus A = 0$

After step 3: `a = B`, `b = A`. Swapped. ✓

Caution: Fails if `a` and `b` point to the **same memory location** (both become 0, since $A \oplus A = 0$).

11.3 Set the k-th Bit

Operation: `x | (1 << k)`

Why: `(1 << k)` creates a mask with only bit `k` set. OR with any value **forces** that bit to 1, leaving all other bits unchanged.

```
x = 1010, k = 0
1 << 0 = 0001
1010 | 0001 = 1011   ← bit 0 is now set ✓
```

11.4 Unset (Clear) the k-th Bit

Operation: `x & ~(1 << k)`

Why: `~(1 << k)` creates a mask with all bits = 1 EXCEPT bit `k` = 0. AND with this mask preserves everything except bit `k`, which is forced to 0.

```
x = 1011, k = 1
1 << 1 = 0010
~0010 = 1101
1011 & 1101 = 1001   ← bit 1 is now cleared ✓
```

11.5 Toggle the k-th Bit

Operation: `x ^ (1 << k)`

Why: XOR with 1 flips the bit. XOR with 0 preserves it. `(1 << k)` has 1 only at position `k`.

```
x = 1011, k = 1
1 << 1 = 0010
1011 ^ 0010 = 1001   ← bit 1 flipped (1→0) ✓

x = 1001, k = 1
1001 ^ 0010 = 1011   ← bit 1 flipped (0→1) ✓
```

11.6 Check if k-th Bit is Set

Operation: `x & (1 << k)`

Why: AND with a mask that has only bit `k` set. If bit `k` in `x` is 1, result is non-zero (2^k). If 0, result is 0.

```
x = 1011, k = 2
1 << 2 = 0100
1011 & 0100 = 0000   ← bit 2 is NOT set ✓

x = 1011, k = 0
1 << 0 = 0001
1011 & 0001 = 0001   ← bit 0 IS set ✓
```

11.7 Remove Rightmost Set Bit

Operation: `x & (x - 1)` — fully proven in §4.1.

11.8 Extract Rightmost Set Bit

Operation: `x & (-x)` — fully proven in §4.2.

Summary Table of Bit Tricks

Operation	Expression	Key Insight
Even/Odd check	`n & 1`	LSB = parity
XOR Swap	3-step XOR	Self-inverse cancellation
Set bit k	`x | (1 << k)`	OR forces bit to 1
Unset bit k	`x & ~(1 << k)`	AND with inverted mask forces to 0
Toggle bit k	`x ^ (1 << k)`	XOR flips the bit
Check bit k	`x & (1 << k)`	AND isolates single bit

Operation	Expression	Key Insight
Remove rightmost 1	`x & (x - 1)`	Borrow ripple + AND
Extract rightmost 1	`x & (-x)`	Two's complement + AND

---

§12 — Time Complexity of Bit Operations

12.1 Why Bit Operations Are O(1)

Modern CPUs process bitwise operations (AND, OR, XOR, NOT, shifts) on entire machine words (32 or 64 bits) in a **single clock cycle** via dedicated hardware (ALU).

This means:

- `a & b` → 1 clock cycle regardless of bit values
- `a | b` → 1 clock cycle
- `a ^ b` → 1 clock cycle
- `a << k` → 1 clock cycle (barrel shifter)

No loops, no carries, pure parallel logic. Each bit is processed independently and simultaneously.

Comparison: Addition appears O(1) too, but internally has carry propagation. Bitwise AND/OR/XOR have NO carry — truly parallel per bit.

12.2 Why Counting Bits via `x & (x - 1)` is O(set bits)

Each application of `x & (x - 1)` removes exactly one set bit (proven in §4.1). So:

- Loop runs exactly K times where K = number of set bits
- Each iteration is $O(1)$ (one AND, one subtraction)
- Total: $O(K)$

For a 32-bit integer: $K \leq 32$. So it's $O(1)$ in practice (bounded by constant), but $O(K)$ is a tighter characterization.

12.3 Complexity Impact in Large Constraints

Scenario	Naive bit-scan	Kernighan's	Impact
32-bit integers, most bits 0	32 iterations always	1-2 iterations	~16× faster
Counting bits for 10^6 numbers	3.2×10^7 ops	~10^7 ops (avg ~10 bits)	3× faster
64-bit sparse values	64 iterations always	1-3 iterations	~20× faster

For bitmask DP:

- $O(2^n \cdot n^2)$ for TSP-style problems
- $O(3^n)$ for submask enumeration over all masks
- These dominate runtime; individual bit operations being O(1) keeps constants small

---

---

§13 — Edge Cases and Common Mistakes

13.1 Zero Handling

`n & (n - 1)` when n = 0:

- `0 - 1` underflows (becomes all 1s in unsigned or −1 in signed)
- `0 & (-1) = 0`
- **Falsely passes** the power-of-two check
- **Fix:** Always guard with `n > 0`

`x & (-x)` when x = 0:

- `-0 = 0`
- `0 & 0 = 0`
- Returns 0 (no rightmost set bit exists) — this is fine, but check for it

XOR of empty set:

- XOR identity is 0, so XOR of nothing = 0
- Starting XOR accumulator at 0 is correct

13.2 Negative Numbers and Two's Complement

Right-shifting negative numbers:

- Arithmetic shift preserves sign (fills with 1s)
- `-1 >> k` = `-1` for all `k` (all bits are 1, shifting in more 1s changes nothing)
- Can cause **infinite loops** if you're shifting until zero

NOT on positive numbers:

- `~5` = `-6` (not `-5`!)
- $\sim x = -(x+1)$
- This catches many people off guard

Using `-x`:

- In two's complement, `-x = ~x + 1`
- For `x = INT_MIN` (e.g., -2^{31}): `-INT_MIN` overflows back to `INT_MIN` (signed overflow is undefined in C/C++)

13.3 Sign Extension

When a **smaller signed type** is widened to a **larger type**, the sign bit is copied into all new positions:

```
8-bit:   1111 1010  (-6 in signed int8)
Extended to 16-bit:
         1111 1111 1111 1010  (-6, sign preserved ✓)
```

The trap in bit manipulation: You expect `0xFA` (250 unsigned), but sign extension gives you `0xFFFA` (−6 signed).

Fix: Mask to strip extension: `value & 0xFF` to keep only the lower 8 bits.

13.4 Integer Overflow

Signed overflow is undefined behavior in C/C++. The compiler may optimize assuming it never happens.

Unsigned overflow is well-defined: wraps modulo 2^n.

When it bites you:

- `1 << 31` on a 32-bit signed int → undefined (sets the sign bit)

- Use `1u << 31` or `1L << 31` for safety
- `INT_MAX + 1` → UB, not just wrong answer

13.5 Shifting Beyond Bit Width

`x << k` or `x >> k` where `k ≥` bit width of `x`:

- In C/C++: **Undefined behavior**
- In Java: `k` is taken modulo 32 (for int) or 64 (for long)
- In Python: No limit (arbitrary precision integers)

Common mistake: `1 << 32` on a 32-bit int doesn't give 2^{32}. It's UB or wraps.

Safe practice: Always ensure `0 ≤ k < bit_width`.

---

§14 — Pattern Recognition Guide

14.1 Signals That a Problem Requires Bit Manipulation

Signal	Likely Technique
"Find the element that appears once / odd number of times"	XOR all elements
"Find missing number"	XOR with expected range
"Find two unique numbers"	XOR + rightmost set bit partition
"O(1) space" constraint	XOR or in-place bit tricks
"Subsets / power set"	Bitmask enumeration
"States that are combinations of items"	Bitmask DP
Problem mentions "XOR" explicitly	Algebraic XOR properties
"Toggle / flip" mentioned	XOR operation
"Maximum XOR pair"	Binary Trie

14.2 Constraint-Based Hints

Constraint	What it hints
$n \leq 16$	Bitmask DP or subset enumeration ($2^{16} = 65536$)
$n \leq 20$	Bitmask DP ($2^{20} \approx 10^6$)
$n \leq 25$	Tight bitmask DP or meet-in-the-middle
$n \leq 40$	Meet-in-the-middle (split into two halves of ~20)
"Power of 2" in constraints	Binary property exploitation
Values up to 10^9	30-bit representation, bit-by-bit processing

14.3 When Bitmask DP is Suitable

Checklist:

- [ ] The state involves "which elements from a set have been processed"

- ☐ The set is small ($n \leq 20$)
- ☐ Order within the set doesn't matter (or can be derived from the mask)
- ☐ Transitions involve adding/removing one element

Classic problems: TSP, Assignment Problem, Hamiltonian Path, optimal game strategy with n items.

14.4 When XOR is the Intended Trick

Checklist:

- ☐ Elements appear in pairs except one/two
- ☐ Problem asks for "difference" or "unique" element
- ☐ O(1) extra space is required
- ☐ Array has a pattern where duplicates cancel out
- ☐ Problem involves ranges `[0, n]` with one missing

If you see "every element appears K times except one that appears once":

- K = 2 → XOR directly
- K = 3 → Bit counting (count each bit position mod 3)
- K even → XOR works
- K odd → Need modular bit counting

---

§15 — Problem-by-Problem: WHY and HOW Every Step Gives the Correct Answer

---

> **Philosophy of this section:** For each problem, we don't just say "use XOR."
> We answer: **WHY does this property apply here? HOW does each step produce the correct answer?**
> Every step is justified with a mathematical reason.

PROBLEM 1: Single Non-Repeating Element

> **Statement:** Array where every element appears **twice** except one. Find the unique element.
> **Constraint:** O(n) time, O(1) space.

Step 1: WHAT do we do?

XOR all elements together. The result is the answer.

Step 2: WHY does this work? (The full reasoning chain)

Observation: We need to "cancel" duplicates. What operation cancels a value with itself?

$$A \oplus A = 0 \quad \text{(Self-Inverse Property)}$$

But wait — the elements are in random order! How can we pair them up?

That's where TWO more properties save us:

$$A \oplus B = B \oplus A \quad \text{(Commutativity — order doesn't matter)}$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) \quad \text{(Associativity — grouping doesn't matter)}$$

These two properties together mean: **no matter what order the elements appear, we can mentally rearrange them into pairs.**

And finally:

$$A \oplus 0 = A \quad \text{(Identity — the survivor remains unchanged)}$$

Step 3: HOW does it give the correct answer? (Trace)

Array: `[4, 1, 2, 1, 2]`

```
Start with xr = 0           (identity element — won't affect result)

xr = 0 ⊕ 4 = 4              (just 4, no pair yet)
xr = 4 ⊕ 1 = 5              (both stored in the XOR)
xr = 5 ⊕ 2 = 7              (three values mixed in)
xr = 7 ⊕ 1 = 6              (the second 1 CANCELS the first 1)
xr = 6 ⊕ 2 = 4              (the second 2 CANCELS the first 2)

Result: 4 ✓
```

But HOW did the 1s cancel even though they weren't adjacent?

Let's see it algebraically:

```
0 ⊕ 4 ⊕ 1 ⊕ 2 ⊕ 1 ⊕ 2

Commutativity lets us reorder:
= 0 ⊕ (1 ⊕ 1) ⊕ (2 ⊕ 2) ⊕ 4

Self-Inverse:
= 0 ⊕ 0 ⊕ 0 ⊕ 4

Identity:
= 4 ✓
```

The key guarantee: XOR doesn't care about position or order. Every pair eventually meets and cancels, no matter how far apart they are in the array. The lone element has no partner to cancel with, so it survives.

WHY XOR specifically (and not AND or OR)?

Operator	a OP a	Problem
AND	a & a = a	Doesn't cancel — duplicate persists
OR	a | a = a	Doesn't cancel — duplicate persists
XOR	**a ⊕ a = 0**	**Cancels perfectly**

XOR is the ONLY bitwise operator where `a OP a = identity element (0)`.

PROBLEM 2: Missing Number in Range [0, n]

> **Statement:** Array of `n` numbers from range `[0, n]` with exactly one missing. Find it.
> **Example:** `n=3`, array = `[3, 0, 1]` → missing = 2

Step 1: WHAT do we do?

XOR all array elements AND XOR all numbers from 0 to n. The result is the missing number.

Step 2: WHY does this work?

The key insight: If we had ALL numbers from 0 to n, and also all array elements, every number would appear EXACTLY TWICE — except the missing number, which appears only ONCE (in the range, but not in the array).

So this reduces to Problem 1!

Formally:

$$\text{result} = \bigoplus_{i=0}^{n} i \quad \oplus \quad \bigoplus_{j=0}^{n-1} \text{arr}[j]$$

For every number that's NOT missing:

- It appears once in `[0, n]` and once in the array → XOR cancels it

For the missing number:

- It appears once in `[0, n]` but NOT in the array → it survives

Step 3: HOW does it give the correct answer? (Trace)

Array: `[3, 0, 1]`, n = 3, missing = 2

```
xr = 0    (start with identity)

Phase 1 — XOR array elements AND indices simultaneously:
  i=0: xr = 0 ⊕ arr[0] ⊕ (0+1) = 0 ⊕ 3 ⊕ 1 = 2
  i=1: xr = 2 ⊕ arr[1] ⊕ (1+1) = 2 ⊕ 0 ⊕ 2 = 0
  i=2: xr = 0 ⊕ arr[2] ⊕ (2+1) = 0 ⊕ 1 ⊕ 3 = 2

Result: xr = 2 ✓
```

Algebraic expansion:

```
(3 ⊕ 0 ⊕ 1) ⊕ (0 ⊕ 1 ⊕ 2 ⊕ 3)

Regroup:
= (0 ⊕ 0) ⊕ (1 ⊕ 1) ⊕ (3 ⊕ 3) ⊕ 2
=    0     ⊕    0     ⊕    0     ⊕ 2
= 2 ✓
```

WHY XOR with indices works: The index range `(i+1)` for `i=0..n-1` gives us `1, 2, ..., n`. Combined with the initial `xr = 0`, we effectively XOR `0, 1, 2, ..., n` — the complete range. No need for a separate loop.

PROBLEM 3: Two Non-Repeating Elements

> **Statement:** Array where every element appears **twice** except TWO elements `x` and `y`. Find both.
> **Example:** `[2, 4, 7, 9, 2, 4]` → answer: 7 and 9

Why Problem 1's approach ISN'T enough

If we XOR everything:

```
2⊕4⊕7⊕9⊕2⊕4 = (2⊕2)⊕(4⊕4)⊕7⊕9 = 0⊕0⊕7⊕9 = 7⊕9
```

We get `xr = 7 ⊕ 9 = 14` (binary `1110`). But this is MIXED — we can't extract `7` and `9` from `14` directly.

Problem: XOR merges two values into one. We need to SEPARATE them.

Step-by-Step Solution with WHY

Step 1: XOR everything → get `xr = x ⊕ y`

WHY: Pairs cancel (proven in Problem 1). Only the two unique values survive, combined via XOR.

```
Array: [2, 4, 7, 9, 2, 4]

xr = 2⊕4⊕7⊕9⊕2⊕4
   = 7⊕9 = 14 = 1110₂
```

Step 2: Find the rightmost set bit of xr

```
number = xr & ~(xr - 1)     OR equivalently     number = xr & (-xr)
```

WHY this specific operation?

`xr = x ⊕ y`. A bit is `1` in xr means `x` and `y` DIFFER at that bit (XOR truth table: different → 1).

We need ANY bit where they differ. The rightmost set bit is the easiest to extract (proven in §4.2/§4.3).

```
xr     = 1110    (14)
xr - 1 = 1101    (13)
~(xr-1)= 0010
xr & ~(xr-1) = 0010    → number = 2 (bit position 1)
```

What does `number = 2` mean? It means at bit position 1, `x` and `y` differ. One of them has a `1` there, the other has a `0`.

```
7 = 0111  → bit 1 = 1
9 = 1001  → bit 1 = 0    ← They differ at bit 1! ✓
```

Step 3: Partition ALL array elements into two groups

Test: `element & number != 0` (bit 1 is set) vs `element & number == 0` (bit 1 is not set)

WHY this partitioning guarantees correctness:

Guarantee A — The two unique numbers land in DIFFERENT groups:

```
7 = 0111  → 0111 & 0010 = 0010 ≠ 0  → Group 1
9 = 1001  → 1001 & 0010 = 0000 = 0  → Group 0
```

They MUST go to different groups because they differ at this bit (that's HOW we chose it).

Guarantee B — Every duplicate pair lands in the SAME group:

```
2 = 0010  → 0010 & 0010 = 0010 ≠ 0  → Group 1
2 = 0010  → 0010 & 0010 = 0010 ≠ 0  → Group 1  ← same group ✓

4 = 0100  → 0100 & 0010 = 0000 = 0  → Group 0
4 = 0100  → 0100 & 0010 = 0000 = 0  → Group 0  ← same group ✓
```

Identical numbers have identical bits → same test result → same group. Always!

Step 4: XOR within each group

```
Group 1 (bit 1 set):  2, 7, 2  →  2⊕7⊕2 = (2⊕2)⊕7 = 7
Group 0 (bit 1 not set): 4, 9, 4  →  4⊕9⊕4 = (4⊕4)⊕9 = 9
```

WHY this final step works: Within each group, duplicates still cancel (same logic as Problem 1). Each group has exactly one unique element, which survives.

The Complete Correctness Chain

```
XOR all → get x⊕y (pairs cancel)
         ↓
  x⊕y ≠ 0 because x ≠ y
         ↓
  At least one bit differs → extract it
         ↓
  That bit separates x from y into different groups
         ↓
  Duplicates stay together (same bits → same group)
         ↓
  XOR per group → one unique per group
         ↓
  Both found ✓
```

PROBLEM 4: Repeating and Missing Number (YOUR QUESTION)

> **Statement:** Array of `n` numbers. Should contain `[1, 2, ..., n]` but one number is **missing** and one number **appears twice**. Find both.
> **Example:** `[1, 3, 3]` → repeating = 3, missing = 2

Why this is HARDER than previous problems

This is NOT a simple "pairs cancel" problem. Here we have:

- Most numbers: appear once in the array AND once in the range `[1,n]` → appear twice total
- The **repeating** number: appears twice in array + once in range → appears THREE times total
- The **missing** number: appears zero times in array + once in range → appears ONCE total

If we XOR array + range, pairs cancel, but the repeating number doesn't fully cancel!

The TRICK: Reduce to Problem 3

Key observation: XOR `(all array elements)` with `(1, 2, ..., n)`.

For a number `k` that's neither missing nor repeating:

- `k` appears **once** in the array and **once** in `{1..n}` → appears **twice** → cancels to 0

For the **repeating** number `R`:

- `R` appears **twice** in the array and **once** in `{1..n}` → appears **3 times** total
- $R \oplus R \oplus R = (R \oplus R) \oplus R = 0 \oplus R = R$ → **R survives**

For the **missing** number `M`:

- `M` appears **zero times** in the array and **once** in `{1..n}` → appears **1 time** total
- → **M survives**

So after XOR:

$$xr = R \oplus M$$

This is EXACTLY the same as Problem 3! We have two unknown values mixed in one XOR result.

Step-by-Step with Full WHY

Array: `[4, 3, 6, 2, 1, 1]`, n = 6. Repeating = 1, Missing = 5.

STEP 1: XOR array elements with range [1, n]

```
xr = 0    (start with identity)

For i = 0:  xr = 0 ⊕ nums[0] ⊕ (i+1) = 0 ⊕ 4 ⊕ 1 = 5
For i = 1:  xr = 5 ⊕ nums[1] ⊕ (i+1) = 5 ⊕ 3 ⊕ 2 = 4
For i = 2:  xr = 4 ⊕ nums[2] ⊕ (i+1) = 4 ⊕ 6 ⊕ 3 = 1
For i = 3:  xr = 1 ⊕ nums[3] ⊕ (i+1) = 1 ⊕ 2 ⊕ 4 = 7
For i = 4:  xr = 7 ⊕ nums[4] ⊕ (i+1) = 7 ⊕ 1 ⊕ 5 = 3
For i = 5:  xr = 3 ⊕ nums[5] ⊕ (i+1) = 3 ⊕ 1 ⊕ 6 = 4
```

Result: xr = 4 = 100_2

WHY this is correct — algebraic proof:

```
xr = (4⊕3⊕6⊕2⊕1⊕1) ⊕ (1⊕2⊕3⊕4⊕5⊕6)
            ↑ array              ↑ range

Regroup by value:
= (1⊕1⊕1) ⊕ (2⊕2) ⊕ (3⊕3) ⊕ (4⊕4) ⊕ (5) ⊕ (6⊕6)
    ↑ 1 appears 3×   ↑ cancel  ↑ cancel ↑ cancel  ↑ only once  ↑ cancel

= 1 ⊕ 0 ⊕ 0 ⊕ 0 ⊕ 5 ⊕ 0
= 1 ⊕ 5
= 4 = 100₂

So xr = Repeating ⊕ Missing = 1 ⊕ 5 = 4 ✓
```

WHY `xr ^ (i+1)` in the loop: We're computing `xr = xr ⊕ nums[i]` (XOR array element) and `xr = xr ⊕ (i+1)` (XOR corresponding range value) in the SAME loop. This is just an optimization — equivalent to XOR-ing array and range separately. The `(i+1)` gives us `1, 2, 3, ..., n` as `i` goes from `0` to `n-1`.

STEP 2: Extract the rightmost set bit

```
number = xr & ~(xr - 1)

xr     = 100
xr - 1 = 011
~(xr-1)= 100
number  = 100 & 100 = 100 = 4 (bit position 2)
```

WHY this step: `xr = R ⊕ M`. A set bit in `xr` means `R` and `M` differ at that position. We need ANY differentiating bit to split them. The rightmost set bit is the simplest to extract.

```
R = 1 = 001  → bit 2 = 0
M = 5 = 101  → bit 2 = 1    ← Different at bit 2 ✓
```

STEP 3: Partition ALL numbers into two groups

We partition BOTH the array elements AND the range [1, n] using the bit test:
`element & number != 0` vs `element & number == 0`

WHY both array AND range?

Remember, we need to separate the repeating number from the missing number. The range [1, n] provides the "expected" copy of every number, including the missing one.

```
number = 4 = 100₂ (testing bit 2)

── Array elements: [4, 3, 6, 2, 1, 1] ──

4 = 100  → 100 & 100 = 100 ≠ 0  → bucket1
3 = 011  → 011 & 100 = 000 = 0  → bucket0
6 = 110  → 110 & 100 = 100 ≠ 0  → bucket1
2 = 010  → 010 & 100 = 000 = 0  → bucket0
1 = 001  → 001 & 100 = 000 = 0  → bucket0
1 = 001  → 001 & 100 = 000 = 0  → bucket0

── Range [1, 6] ──

1 = 001  → 001 & 100 = 000 = 0  → bucket0
2 = 010  → 010 & 100 = 000 = 0  → bucket0
3 = 011  → 011 & 100 = 000 = 0  → bucket0
4 = 100  → 100 & 100 = 100 ≠ 0  → bucket1
5 = 101  → 101 & 100 = 100 ≠ 0  → bucket1
6 = 110  → 110 & 100 = 100 ≠ 0  → bucket1
```

XOR within each bucket:

```
bucket0 = 3 ⊕ 2 ⊕ 1 ⊕ 1 ⊕ 1 ⊕ 2 ⊕ 3
           ↑ from array    ↑from range
        = (1⊕1⊕1) ⊕ (2⊕2) ⊕ (3⊕3)
        = 1 ⊕ 0 ⊕ 0
        = 1

bucket1 = 4 ⊕ 6 ⊕ 4 ⊕ 5 ⊕ 6
           ↑array    ↑range
        = (4⊕4) ⊕ (6⊕6) ⊕ 5
        = 0 ⊕ 0 ⊕ 5
        = 5
```

Result: One bucket gives `1`, the other gives `5`. These are the repeating and missing numbers.

STEP 4: Which one is repeating and which is missing?

We can't tell from XOR alone. A simple scan of the array tells us:

- Check if `1` appears in the array → Yes, it's the **repeating** number
- Then `5` is the **missing** number

(Or: count occurrences of either result in the array.)

WHY the Partitioning is Guaranteed Correct

Let's prove the two critical guarantees:

Guarantee 1: R and M are in DIFFERENT buckets

`xr = R ⊕ M` has a `1` at the chosen bit position. By definition of XOR, R and M differ at this bit. One has `0`, the other has `1`. So the `& number` test sends them to different buckets.

Guarantee 2: Every "normal" number cancels within its bucket

A normal number `k` (neither repeating nor missing) appears exactly **once** in the array and **once** in the range. Both copies have the SAME bits → both go to the SAME bucket. Within the bucket: $k \oplus k = 0$.

Guarantee 3: The repeating and missing numbers survive

- `R` appears **twice** in array + **once** in range = **3 times** in its bucket. Since $R \oplus R \oplus R = R$, it survives.
- `M` appears **zero times** in array + **once** in range = **1 time** in its bucket. It survives trivially.

Together: Each bucket ends up with exactly one survivor — R or M.

The Complete Correctness Chain for Repeating & Missing

```
XOR(array) ⊕ XOR(range) = R ⊕ M
    ↓ WHY? Because every normal number appears exactly 2× and cancels

R ⊕ M ≠ 0 because R ≠ M
    ↓ WHY? They're defined as different numbers

Extract rightmost set bit → a bit where R and M differ
    ↓ WHY? XOR = 1 at a position means different bits

Partition array AND range by this bit
    ↓ WHY? Separates R from M (different bits)
    ↓ WHY safe? Normal numbers: both copies go same bucket (same bits)

XOR within each bucket → survivors are R and M
    ↓ WHY? Normal pairs cancel; R appears odd times (3); M appears once

One final check tells you which is repeating vs missing ✓
```

PROBLEM 5: Single Number Among Triples

> **Statement:** Every element appears **3 times** except one that appears **once**. Find it.
> **Constraint:** O(n) time, O(1) space.

WHY XOR alone DOESN'T work here

$$A \oplus A \oplus A = (A \oplus A) \oplus A = 0 \oplus A = A \neq 0$$

Three copies DON'T cancel with XOR! Only pairs do ($A \oplus A = 0$). So XOR gives garbage.

The KEY Insight: Bit Counting mod 3

Approach: For EACH bit position, count how many numbers have a `1` at that position.

- Numbers appearing 3 times contribute `0` or `3` to the count at each position
- `3 mod 3 = 0` → their contribution vanishes
- The unique number contributes `1` to positions where it has a `1`
- After taking `count mod 3`, only the unique number's bits remain

HOW it works (Trace)

Array: `[5, 3, 5, 5, 3, 3, 8]` → Unique = 8 = `1000₂`

```
Count 1s at each bit position:

Bit 3:  5=0, 3=0, 5=0, 5=0, 3=0, 3=0, 8=1 → count = 1 → 1 mod 3 = 1
Bit 2:  5=1, 3=0, 5=1, 5=1, 3=0, 3=0, 8=0 → count = 3 → 3 mod 3 = 0
Bit 1:  5=0, 3=1, 5=0, 5=0, 3=1, 3=1, 8=0 → count = 3 → 3 mod 3 = 0
Bit 0:  5=1, 3=1, 5=1, 5=1, 3=1, 3=1, 8=0 → count = 6 → 6 mod 3 = 0

Result bits: 1 0 0 0 = 8 ✓
```

WHY mod 3?

If a number `x` appears `K` times and has a `1` at bit position `p`:

- It contributes `K` to the count at position `p`

- `K mod 3 = 0` when `K = 3` (the triples) → contribution disappears
- `K mod 3 = 1` when `K = 1` (the unique) → the `1` survives

For K times (generalization):

- If others appear **K times** and one appears **once**: count each bit position mod K
- This works for any K ≥ 2

WHY O(1) space?

We only need a fixed number of counters (32 counters for 32-bit integers). That's constant space regardless of array size.

PROBLEM 6: Count Number of Set Bits

> **Statement:** Count the number of `1` bits in a number.

WHY `x & (x-1)` is the correct tool

We need something that:

1. Removes exactly one `1` bit per step (so we can count)
2. Terminates when no `1` bits remain (x becomes 0)
3. Doesn't touch other bits (so count stays accurate)

`x & (x-1)` does ALL three (proven in §4.1 — 3-zone analysis).

HOW it gives the correct count (WHY each step)

x = 52 = 110100_2 (3 set bits)

```
ITERATION 1:
  WHY does it work? Borrow ripple flips rightmost 1 and trailing 0s.

  x     = 110100
  x-1   = 110011     ← pivot (position 2) flips, trailing 0s become 1s
  x&(x-1)= 110000    ← pivot killed, prefix preserved, suffix zeroed
  count = 1

  Bit removed: position 2 ✓
  Remaining 1s: positions 4 and 5

ITERATION 2:
  x     = 110000
  x-1   = 101111     ← pivot now at position 4
  x&(x-1)= 100000    ← position 4 killed
  count = 2

  Remaining 1s: position 5

ITERATION 3:
  x     = 100000
  x-1   = 011111     ← pivot now at position 5
  x&(x-1)= 000000    ← position 5 killed
  count = 3

  x = 0 → STOP. Answer: 3 ✓
```

WHY it stops at exactly the right time: When all `1` bits are removed, `x = 0`. The loop condition `x != 0` exits. The count equals the number of `1` bits because each iteration removes exactly one.

WHY it's faster than checking each bit: We skip all-zero regions. If x = `10000000000000000000000000000001`, the naive scan checks all 32 positions. Kernighan's does only 2 iterations.

PROBLEM 7: Check if Number is Power of Two

> **Statement:** Is `n` a power of 2?

WHY `n & (n-1) == 0` is the correct test

What makes a power of 2 special in binary?

2^k has exactly ONE `1` bit (at position `k`). Every other number has 2+ set bits.

What does `n & (n-1)` do? Removes the rightmost set bit (proven in §4.1).

The logic:

- If n has **exactly 1** set bit: removing it → `0`. Test passes.
- If n has **2+** set bits: removing one → still non-zero. Test fails.

This is not a coincidence — it's a direct consequence of `x & (x-1)` removing **exactly one** bit.

HOW to handle edge cases

n = 0: `0 & (-1) = 0`, which would falsely pass. But 0 is NOT 2^k for any integer k.

WHY 0 is a false positive: `x & (x-1)` removes the rightmost set bit. Zero has no set bits to remove, but the result is still 0 (the `& -1` is a red herring). So the test `== 0` trivially passes.

Fix: Guard with `n > 0`:

$$n > 0 \quad \wedge \quad n \,\&\, (n-1) = 0$$

PROBLEM 8: XOR Swap

> Swap two variables without a temp variable.

WHY this works (Algebraic proof)

```
Let original: a = A, b = B

Step 1:  a = A ⊕ B         b = B
Step 2:  a = A ⊕ B         b = (A ⊕ B) ⊕ B = A ⊕ (B ⊕ B) = A ⊕ 0 = A
Step 3:  a = (A ⊕ B) ⊕ A   b = A
         = A ⊕ A ⊕ B
         = 0 ⊕ B
         = B
```

WHY Step 2 recovers A: `(A ⊕ B) ⊕ B` — the `B` cancels with itself (Self-Inverse), leaving `A`.

WHY Step 3 recovers B: `(A ⊕ B) ⊕ A` — the `A` cancels with itself, leaving `B`.

The deep insight: `A ⊕ B` is a **reversible encoding** of both values. XOR-ing with either original value extracts the other.

WHY it fails when a and b are the same memory location

If `&a == &b` (same address), then after Step 1:

```
a = A ⊕ A = 0    (but b points to same memory, so b = 0 too!)
```

Everything becomes 0. Both values are destroyed.

PROBLEM 9: Maximum XOR Pair (Trie-based)

> Find two elements in array whose XOR is maximum.

WHY greedy works (and brute force doesn't)

Brute force: Try all $O(n^2)$ pairs. Too slow for large arrays.

Key mathematical fact (MSB Dominance):

$$2^k > 2^{k-1} + 2^{k-2} + \cdots + 2^0 = 2^k - 1$$

What this means for XOR maximization:

Getting a `1` at bit position `k` in the XOR result is MORE VALUABLE than getting `1` at ALL positions below `k` combined.

WHY this makes greedy optimal: If we have a choice:

- Option A: Get `1` at bit 5 (value = 32)
- Option B: Get `1` at bits 4,3,2,1,0 (value = 16+8+4+2+1 = 31)

Option A wins. Always. So we process bits from the highest to the lowest, and at each bit, we greedily try to make the XOR `1`.

HOW the Trie enables this

Without Trie: For each number `x`, we'd scan the entire array to find which element maximizes `x ⊕ element`. That's $O(n)$ per query.

With Trie: We insert all numbers bit by bit (MSB first) into a binary tree. For a query number `x`:

```
At each level (bit position k, going from MSB to LSB):
  x has bit b at this position
  To make XOR = 1, we need the OPPOSITE bit (1-b)

  If the trie has a child with bit (1-b):
      → Go there! XOR bit = 1 at this position ✓
  Else:
      → Go to child with bit b. XOR bit = 0 at this position
```

WHY this gives the maximum:

- At each level, we try the best option first (opposite bit → XOR = 1)
- MSB dominance guarantees no lower-level choice can overcome a bad high-level choice
- So the greedy path IS the optimal path
- No backtracking ever needed

Trace Example

Array: `[3, 10, 5, 25, 2]`, binary (5 bits):

```
 3 = 00011
10 = 01010
 5 = 00101
25 = 11001
 2 = 00010
```

Query: Find max XOR with `x = 5 = 00101`

```
Bit 4: x=0, want opposite=1. Trie has 1? (from 25=11001) YES → go to 1
Bit 3: x=0, want opposite=1. Trie has 1 from this path? (25: bit3=1) YES → go to 1
Bit 2: x=1, want opposite=0. Trie: 25 has bit2=0. YES → go to 0
Bit 1: x=0, want opposite=1. Trie: 25 has bit1=0. NO. → go to 0
Bit 0: x=1, want opposite=0. Trie: 25 has bit0=1. NO. → go to 1

Matched number: 11001 = 25
XOR: 5 ⊕ 25 = 00101 ⊕ 11001 = 11100 = 28

This is the maximum possible ✓
```

PROBLEM 10: Subset Generation with Bitmask

> Generate all subsets of a set of `n` elements.

WHY bitmasks represent subsets

An `n`-element set has 2^n subsets. An `n`-bit integer also has 2^n possible values (0 to $2^n - 1$). There's a **natural bijection**:

- Bit `i` = 1 → element `i` is IN the subset
- Bit `i` = 0 → element `i` is NOT in the subset

WHY this mapping is correct: Each element has 2 independent choices (in/out). A bit has 2 independent states (1/0). The multiplication principle gives 2^n for both.

HOW to extract which elements are in a mask

For mask `m`, check each bit:

- `m & (1 << i) != 0` → element `i` is in the subset

WHY `(1 << i)` works as a probe: `(1 << i)` has a single `1` at position `i`. AND with `m` isolates bit `i`. Non-zero iff bit `i` was set.

Trace Example

Set = `{a, b, c}`, n = 3.

```
mask = 5 = 101₂

Check bit 0:  101 & 001 = 001 ≠ 0  → a ∈ subset ✓
Check bit 1:  101 & 010 = 000 = 0  → b ∉ subset
Check bit 2:  101 & 100 = 100 ≠ 0  → c ∈ subset ✓

Subset for mask 5: {a, c} ✓
```

PROBLEM 11: Bitmask DP (State Compression)

> **Example:** TSP — visit all cities exactly once with minimum cost.

WHY bitmask replaces a boolean array

A state "which cities visited" needs to track n yes/no values. Two representations:

Approach	Representation	Compare cost	Hash cost	Memory
Boolean array	`[T, F, T, T, F]`	$O(n)$	$O(n)$	$O(n)$ per state
Bitmask	`10110` = 22	**O(1)**	**O(1)**	**O(1)** per state

WHY O(1) comparison: Two integers are compared in a single CPU instruction. Two arrays need element-by-element comparison.

HOW transitions work and WHY they're correct

State: `DP[mask][i]` = minimum cost to have visited exactly the cities in `mask`, ending at city `i`.

Transition — add unvisited city `j`:

```
1. Check if j is unvisited:  mask & (1 << j) == 0

   WHY: (1 << j) has a 1 at position j. AND with mask isolates
   bit j. If 0, city j is not in the "visited" set.

2. New mask:  new_mask = mask | (1 << j)

   WHY: OR with (1 << j) sets bit j to 1 — "add city j to visited set."
   All other bits unchanged — previously visited cities stay visited.

3. Update: DP[new_mask][j] = min(DP[new_mask][j], DP[mask][i] + cost[i][j])

   WHY: We came from city i (cost DP[mask][i]) and travel to j (cost cost[i][j]).
   Take minimum over all possible "previous cities" i.
```

WHY this covers all valid paths: Every mask from `0` to $2^n - 1$ represents a valid subset of visited cities. Every bit transition `mask → mask | (1 << j)` represents visiting one new city. By iterating masks in increasing order, we ensure all sub-paths are computed before they're needed.

PROBLEM 12: Check / Set / Unset / Toggle k-th Bit

WHY each operation uses its specific operator

The fundamental principle: We need an operation that affects ONLY bit `k` and leaves all other bits unchanged.

Creating the tool — `(1 << k)`:

```
1 << k = 000...010...0
                ↑
            only bit k is 1
```

Now:

Goal	Operation	WHY this operator?
Set bit k	`x | (1 << k)`	OR truth table: `0|1=1, 1|1=1` → forces bit to 1. `0|0=0, 1|0=1` → other bits unchanged (OR with 0 = identity)
Unset bit k	`x & ~(1 << k)`	`~(1 << k)` = all 1s except position k. AND truth table: `x&1=x` (identity), `x&0=0` (forces to 0). So all bits preserved except bit k → forced to 0
Toggle bit k	`x ^ (1 << k)`	XOR truth table: `x^1=~x` (flips), `x^0=x` (identity). So bit k flips, all others unchanged
Check bit k	`x & (1 << k)`	AND isolates bit k. All other bits AND with 0 → become 0. Bit k AND with 1 → stays as-is. Non-zero means it was 1

The unifying insight

Each operator has a special "identity" value and a special "active" value:

Operator	Identity (no change)	Active value	Active effect
OR	0	1	Forces to 1
AND	1	0	Forces to 0
XOR	0	1	Flips

We construct `(1 << k)` or `~(1 << k)` to place the "active" value at bit `k` and "identity" values everywhere else. That's why ONLY bit `k` is affected.

PROBLEM 13: Even/Odd Check

WHY `n & 1` works

The mathematical fact: Any integer n can be written as:

$$n = b_k \cdot 2^k + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

Every term except $b_0 \cdot 2^0$ is a multiple of 2 (because 2^i for $i \geq 1$ is even).

So: $n \mod 2 = b_0$

`b_0` is the LSB. `n & 1` extracts the LSB.

```
n & 1 = 0 → n is even (LSB = 0, no odd component)
n & 1 = 1 → n is odd  (LSB = 1, has the 2⁰ = 1 component)
```

WHY `& 1` isolates the LSB: `1 = 000...001`. AND with this mask zeroes every bit except position 0. Only the LSB survives.

SUMMARY: The Problem → Property → Why Chain

Problem	Property Used	WHY It Works
Single unique (pairs cancel)	$A \oplus A = 0$, commutativity, associativity	Pairs cancel regardless of order; unique survives as identity
Missing number	XOR array $\oplus$ XOR range	Reduces to single-unique: present numbers appear 2× and cancel
Two unique numbers	XOR + rightmost set bit partition	XOR gives combined value; differentiating bit separates them; duplicates always go same bucket
Repeating & Missing	XOR array $\oplus$ XOR range + partition	Reduces to two-unique: normal numbers cancel (appear 2×); R survives (3×→odd); M survives (1×→odd)
Unique among triples	Bit counting mod 3	Triples contribute multiples of 3 at each bit → vanish mod 3; unique's bits survive
Count set bits	`x & (x-1)` loop	Each step kills exactly 1 bit (3-zone proof); count = iterations
Power of 2	`n & (n-1) == 0`	Single-bit number → removing it leaves 0; multi-bit → still non-zero
Max XOR pair	Trie + greedy MSB-first	MSB dominance ($2^k > \sum_{j<k} 2^j$) → greedy is optimal
Subset gen	Mask bits ↔ elements	Bijection: 2^n masks ↔ 2^n subsets
Bitmask DP	Integer = state	$O(1)$ compare/hash/transition vs $O(n)$ for arrays
k-th bit ops	`(1 << k)` + appropriate operator	Each operator has identity + active value; mask places active only at bit k
Even/Odd	`n & 1`	LSB = $n \mod 2$ because all higher terms are even multiples

---

Quick Reference Card

```
┌──────────────────────────────────────────────────────────┐
│                   BIT MANIPULATION CHEAT SHEET           │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  FUNDAMENTALS                                            │
│  ─────────────                                           │
│  Position i has weight 2^i                               │
│  -x = ~x + 1  (two's complement)                        │
│  ~x = -(x+1)                                             │
│  Signed n-bit range: [-2^(n-1), 2^(n-1) - 1]            │
│  Unsigned n-bit range: [0, 2^n - 1]                      │
│                                                          │
│  SHIFT = MULTIPLY/DIVIDE                                 │
│  ──────────────────────                                  │
│  x << k  =  x × 2^k                                     │
│  x >> k  =  ⌊x / 2^k⌋   (positive numbers)             │
│                                                          │
│  CORE IDENTITIES                                         │
│  ────────────────                                        │
│  x & (x-1)   → remove rightmost set bit                 │
│  x & (-x)    → isolate rightmost set bit                │
│  x & ~(x-1)  → same as x & (-x)                         │
│  n & (n-1)==0 → power of 2  (guard: n > 0)              │
│                                                          │
│  XOR PROPERTIES                                          │
│  ──────────────                                          │
│  A ⊕ A = 0        A ⊕ 0 = A                              │
│  Commutative, Associative                                │
│  X ⊕ Y ⊕ X = Y   (cancellation)                          │
│                                                          │
│  k-TH BIT OPERATIONS                                     │
│  ────────────────────                                    │
│  Set:    x | (1 << k)                                    │
│  Unset:  x & ~(1 << k)                                   │
│  Toggle: x ^ (1 << k)                                    │
│  Check:  x & (1 << k)                                    │
│                                                          │
│  BITMASK DP                                              │
│  ──────────                                              │
│  n ≤ 20 → 2^n states feasible                            │
│  Submask enum: s = (s-1) & m, complexity O(3^n)          │
│  Union: A|B  Intersection: A&B  Difference: A & ~B      │
│                                                          │
│  COMPLEXITY                                              │
│  ──────────                                              │
│  Single bitwise op: O(1)                                 │
│  Kernighan's popcount: O(set bits)                       │
│  Bitmask DP: O(2^n × n²) for TSP-style                  │
│  Trie XOR query: O(word_size) per query                  │
│                                                          │
│  EDGE CASES                                              │
│  ──────────                                              │
│  Zero: guard n > 0 for power-of-2 check                  │
│  Negative right-shift: sign extension (fills with 1s)    │
│  ~x of positive → negative (not just flip of value)      │
│  Shift ≥ bit_width: undefined behavior (C/C++)           │
│                                                          │
│  INTERVIEW SIGNALS                                       │
│  ──────────────────                                      │
│  n ≤ 20 → bitmask DP                                     │
│  "appears twice except one" → XOR                        │
│  "O(1) space" → bit tricks                               │
│  "maximum XOR" → binary trie                             │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

End of Notes.